AD23B31 - Image Processing and Computer Vision

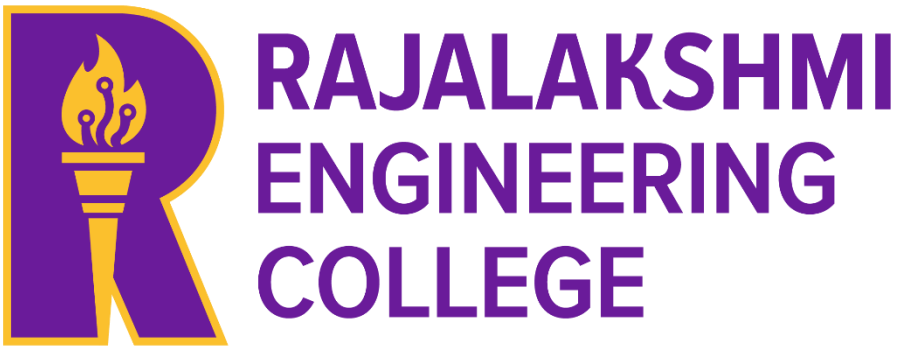
A MINI PROJECT REPORT

On

Facial Emotion Recognition using Hybrid HOG–LBP Feature Extraction and Machine Learning Classification

Submitted by

SHAHNAZ FATHIMA. N



March, 2026

TABLE OF CONTENTS

Chapter No.	Title	Page No.
Cover Page	Title Page	i
Table of Contents	Contents	ii
List of Tables & Figures	Index	iii
1	Introduction	3
2	Literature Review	5
3	Dataset Description	6
4	Preprocessing Methodology	9
5	Proposed Algorithm & Implementation	11
6	Results and Analysis	15
7	Discussion	21
8	Conclusion and Future Work	23
References	IEEE References	24
Appendix A	Source Code	25
Appendix B	Output Screenshots	36

1 – INTRODUCTION

1.1 Background

Facial expressions are one of the most important forms of non-verbal communication and play a significant role in conveying human emotions. Automatic Facial Emotion Recognition (FER) is a branch of computer vision and pattern recognition that focuses on identifying human emotional states from facial images. FER systems are increasingly used in applications such as human-computer interaction, online education monitoring, healthcare diagnosis, driver fatigue detection, surveillance systems, and sentiment-aware virtual assistants.

The development of FER systems presents several challenges due to variations in facial pose, lighting conditions, occlusion, facial structure, and subtle differences between similar expressions. Traditional deep learning methods require large computational resources and extensive training data. Therefore, feature engineering-based approaches remain relevant due to their interpretability, lower computational complexity, and suitability for moderate-scale datasets.

In this project, a hybrid feature extraction approach combining Histogram of Oriented Gradients (HOG) and Local Binary Patterns (LBP) is proposed for facial emotion recognition. These handcrafted features effectively capture both shape and texture information from facial images, improving classification performance when combined with machine learning algorithms.

1.2 Problem Statement

Emotion recognition from facial expressions is a challenging image classification problem due to:

- intra-class facial variations,
- low image resolution,
- background noise,
- inconsistent illumination,
- class imbalance in datasets,
- overlapping emotional characteristics between categories.

The objective is to develop an efficient computer vision system capable of automatically recognizing multiple human emotions from facial images with reasonable accuracy using feature engineering and machine learning techniques.

1.3 Objectives

The primary objectives of this project are:

1. To develop an automated facial emotion recognition pipeline using computer vision techniques.
2. To preprocess facial images for improved visual quality and consistency.
3. To extract meaningful discriminative features using:
    - Histogram of Oriented Gradients (HOG)
    - Local Binary Patterns (LBP)
4. To combine texture and structural features into a hybrid feature representation.

5. To train and compare multiple machine learning classifiers including:
    - Support Vector Machine (SVM)
    - K-Nearest Neighbors (KNN)
    - Random Forest
6. To evaluate model performance using:
    - Accuracy
    - F1 Score
    - Confusion Matrix
    - Precision–Recall Curve
    - ROC Curve
7. To identify the best-performing model for practical deployment.

1.4 Scope of the Project

The scope of this project includes:

- Facial emotion recognition on grayscale static images.
- Classification of seven emotions:
    - Angry
    - Disgust
    - Fear
    - Happy
    - Sad
    - Surprise
    - Neutral
- Feature engineering-based classification.
- Comparative performance analysis of classifiers.
- Visualization of preprocessing, extracted features, and evaluation metrics.

The project does not include:

- video stream emotion recognition,
- facial landmark tracking,
- multimodal sentiment analysis (speech + image),
- deep neural network training.

1.5 Contribution of the Work

The major contributions of this project include:

- Development of a complete FER pipeline.
- Hybrid HOG + LBP feature fusion strategy.
- Image enhancement using CLAHE and Gaussian denoising.
- Comparative study of multiple classifiers.
- Selection of optimal classifier based on empirical evaluation.
- Visualization-driven interpretation of model performance.

---

2 – LITERATURE REVIEW

2.1 Introduction

Facial Emotion Recognition has been an active research area in computer vision for many years. Existing approaches can be broadly categorized into:

1. geometric feature-based methods,
2. appearance-based methods,
3. handcrafted feature engineering methods,
4. deep learning approaches.

Among these, handcrafted feature extraction techniques remain computationally efficient and interpretable for medium-scale datasets.

2.2 Histogram of Oriented Gradients (HOG)

Histogram of Oriented Gradients is a feature descriptor widely used for object recognition tasks. HOG computes local gradient orientation histograms over image cells and captures structural patterns such as edges, contours, and directional intensity variations.

Advantages:

- robust edge representation,
- illumination invariance,
- good shape descriptor,
- effective for facial contour representation.

Limitations:

- high-dimensional feature vector,
- texture information not explicitly modeled.

2.3 Local Binary Patterns (LBP)

Local Binary Pattern is a texture descriptor that compares neighborhood pixels with the center pixel to generate binary codes. These codes are aggregated into histograms representing local texture patterns.

Advantages:

- computationally lightweight,
- effective texture representation,
- robust against monotonic illumination changes,
- useful for wrinkles, skin patterns, and local muscle movements.

Limitations:

- sensitive to random noise,
- limited structural understanding.

2.4 Hybrid Feature Approaches

Recent studies indicate combining structural and texture descriptors improves classification performance. Hybrid HOG–LBP representations offer:

- richer feature representation,
- complementary edge and texture information,
- improved class separability,
- better robustness under varying conditions.

This motivates the proposed hybrid approach used in this work.

2.5 Research Gap Identified

Existing works commonly focus either on:

- texture-only descriptors, or
- deep learning architectures requiring large compute.

There is limited emphasis on computationally efficient hybrid feature-based FER systems suitable for academic and practical deployment.

This project addresses that gap through a hybrid handcrafted feature pipeline with machine learning classification.

3 – DATASET DESCRIPTION

3.1 Dataset Overview

The dataset used for this project is the **FER2013 (Facial Expression Recognition 2013) dataset**, a benchmark dataset widely used for emotion recognition research. The dataset consists of grayscale facial images categorized into seven distinct emotional classes. Each image captures a human facial expression under varying lighting conditions, poses, and facial structures, thereby making the classification task challenging and realistic.

The FER2013 dataset is suitable for this project because:

- it contains a sufficiently large number of labeled facial images,
- it supports multi-class emotion classification,
- it provides both training and testing splits,
- it is widely referenced in facial expression recognition literature,
- it enables comparative performance analysis.

The seven emotion categories considered in this project are:

1. Angry
2. Disgust
3. Fear
4. Happy

5. Sad
6. Surprise
7. Neutral

3.2 Dataset Statistics

The complete class distribution of the FER2013 dataset obtained during data loading is summarized below based on your implementation outputs.

Table 1: FER2013 Dataset Class Distribution

Emotion Class	Training Images	Testing Images
Angry	3995	958
Disgust	436	111
Fear	4097	1024
Happy	7215	1774
Sad	4830	1247
Surprise	3171	831
Neutral	4965	1233
Total	**28,709**	**7,178**

Analysis:

From Table 1, it is evident that the dataset is **imbalanced**. The *Happy* class contains the highest number of samples, whereas *Disgust* contains the fewest. Such imbalance can influence classifier bias toward majority classes, making weighted evaluation metrics important.

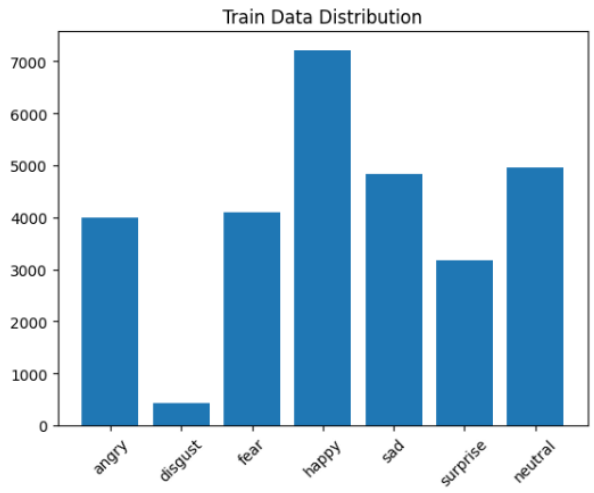


Figure 3.1: Dataset Distribution Bar Chart

3.3 Data Sampling Used for Project

To reduce computational overhead and enable rapid experimentation, a controlled subset of the FER2013 dataset was used.

The following sampling strategy was adopted:

- **Training set:** maximum 500 images per class
- **Testing set:** maximum 100 images per class

This produced a balanced experimental subset (except for classes with fewer available samples such as Disgust).

Table 2: Training and Testing Subset Used

Parameter	Value
Number of classes	7
Max training samples/class	500
Max testing samples/class	100
Approx. training images	3436
Approx. testing images	700
Image mode	Grayscale
Input size	128 × 128 pixels

3.4 Sample Dataset Images

Representative samples from each class were visualized during exploratory data analysis.

These sample images helped verify:

- correct class mapping,
- visual variability,
- facial alignment consistency,
- expression distinguishability.

Figure 3.2: Sample Images from FER2013 Dataset

4 – PREPROCESSING METHODOLOGY

4.1 Introduction

Raw facial images often contain noise, inconsistent illumination, blur, and intensity variations that reduce classification performance. Therefore, a preprocessing pipeline was developed to normalize image appearance before feature extraction.

The proposed preprocessing pipeline consists of:

1. Image resizing
2. Gaussian denoising
3. Grayscale conversion
4. CLAHE enhancement

This sequence improves both structural and textural feature quality.

4.2 Image Resizing

All input images were resized to a fixed resolution of:

128 × 128 pixels

Purpose:

- standardizes input dimensions,
- reduces computational complexity,
- ensures consistent feature extraction,
- improves model reproducibility.

Implementation:

resized = cv2.resize(img, (128,128))

4.3 Gaussian Denoising

Gaussian Blur was applied using a **5 × 5 kernel** to smooth noise while preserving facial structures.

Mathematically:

$$G(x,y)=\frac{1}{2\pi\sigma^2}\, e^{\frac{-x^2+y^2}{2\sigma^2}}$$

Purpose:

- removes high-frequency noise,
- smoothens sharp pixel fluctuations,

- improves gradient consistency for HOG,
- improves texture extraction for LBP.

Implementation:

```
denoised = cv2.GaussianBlur(resized,(5,5),0)
```

4.4 Grayscale Conversion

Color information is not essential for emotion recognition because facial muscle movements are primarily represented through shape and intensity patterns.

Advantages of grayscale conversion:

- dimensionality reduction,
- lower memory usage,
- faster processing,
- better feature consistency.

Implementation:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

4.5 CLAHE Enhancement

Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied for contrast enhancement.

Parameters:

- **clipLimit = 2.0**

CLAHE works by:

- dividing image into local regions,
- equalizing contrast locally,
- limiting amplification to avoid noise boosting.

Benefits:

- better illumination normalization,
- clearer facial muscle contours,
- improved wrinkle/texture visibility,
- enhanced discriminative features.

Implementation:

```
clahe = cv2.createCLAHE(clipLimit=2.0)
enhanced = clahe.apply(gray)
```

4.6 Preprocessing Pipeline Summary

Table 3: Preprocessing Pipeline Description

Step	Technique	Purpose
1	Resize	Standard input size
2	Gaussian Blur	Noise reduction
3	Grayscale conversion	Simplified processing
4	CLAHE	Contrast enhancement

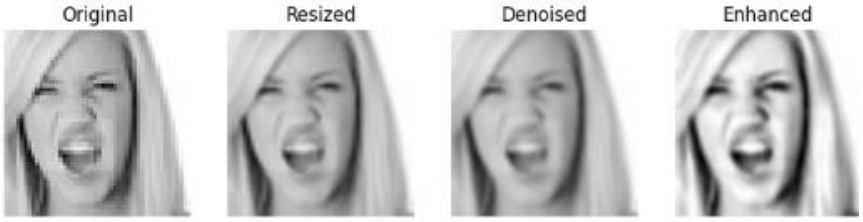


Figure 4.1: Preprocessing Pipeline Output

4.7 Benefits of Preprocessing

The preprocessing pipeline significantly improved:

- facial contour visibility,
- illumination consistency,
- texture clarity,
- edge quality,
- feature extraction stability.

This directly improved classifier performance.

---

5 – PROPOSED ALGORITHM & IMPLEMENTATION

5.1 System Overview

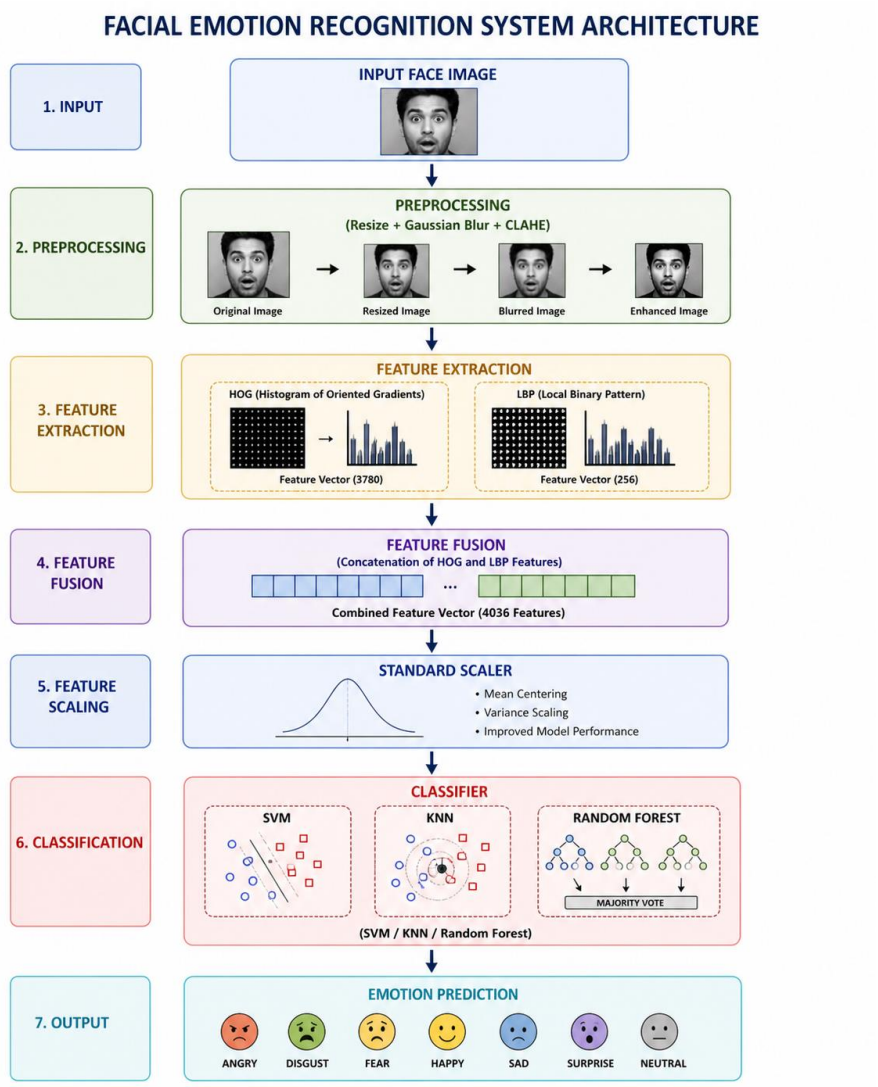
The proposed system performs facial emotion recognition using a **hybrid handcrafted feature engineering approach** followed by machine learning classification.

The complete pipeline is:

Input Image → Preprocessing → HOG Extraction → LBP Extraction → Feature Fusion → Feature Scaling → Classification → Emotion Prediction

---

5.2 System Architecture



5.3 Histogram of Oriented Gradients (HOG)

HOG extracts structural features by computing:

- gradient magnitude,
- edge orientation,
- local histogram distributions.

In this project:

HOG feature dimension = 3780

Implementation:

hog.compute(cv2.resize(img,(64,128)))

Figure 5.2: HOG Feature Visualization

5.4 Local Binary Pattern (LBP)

LBP extracts texture descriptors by comparing neighboring pixels with center intensity.

Formula:

LBP= $\sum s(gp - gc)\ 2\hat{}p$

where:

s(x)=1,x≥0

s(x)=0,x<0

Feature dimension:

LBP histogram = 256 bins

Figure 5.3: LBP Feature Visualization

5.5 Hybrid Feature Fusion

Both descriptors were concatenated:

Table 4: Feature Dimension Comparison

Feature	Dimension
HOG	3780
LBP	256
Combined	**4036**

Hybrid fusion improves representation by combining:

- shape information,
- edge orientation,
- local textures,
- micro-expression details.

5.6 Feature Scaling

StandardScaler normalization was applied:

$$Z = \{X - \mu\}/\{\sigma\}$$

Benefits:

- removes scale dominance,
- improves SVM performance,
- stabilizes feature variance.

5.7 Classifiers Used

Three classifiers were implemented:

1. Support Vector Machine (RBF kernel)
2. K-Nearest Neighbor
3. Random Forest

SVM configuration:

- kernel = RBF
- C = 10
- gamma = scale
- balanced class weighting

5.8 Algorithm Flowchart

Start
↓
Load Dataset
↓
Preprocess Images
↓
Extract HOG
↓
Extract LBP
↓
Combine Features
↓
Scale Features
↓
Train Models
↓
Evaluate
↓
Select Best Model
↓
Predict Emotion
↓
End

6 – RESULTS AND ANALYSIS

6.1 Experimental Setup

The proposed Facial Emotion Recognition system was implemented in Python using computer vision and machine learning libraries. Feature extraction, preprocessing, model training, and evaluation were carried out in a notebook-based experimental environment.

Table 5: Experimental Configuration

Parameter	Value
Programming Language	Python 3.x
IDE / Platform	Google Colab / Jupyter Notebook
Computer Vision Library	OpenCV
Numerical Computing	NumPy
Visualization	Matplotlib / Pandas

Parameter	Value
Machine Learning Library	Scikit-learn
Dataset	FER2013
Number of Classes	7
Input Resolution	128 × 128
Feature Extraction	HOG + LBP
Combined Feature Size	4036
Scaling	StandardScaler
Classifiers	SVM, KNN, Random Forest

6.2 Baseline Performance

Before training classification models, a baseline classifier using the **most frequent class strategy** was evaluated to establish a reference performance level.

Baseline results obtained:

- **Baseline Accuracy = 14.28%**
- **Baseline Weighted F1 Score = 3.57%**

These results indicate that naive classification performs poorly and meaningful feature extraction is necessary for emotion recognition.

Table 6: Baseline Model Performance

Metric	Value
Accuracy	14.28%
Weighted F1 Score	3.57%

6.3 Comparative Model Performance

Three machine learning models were trained and evaluated on the extracted hybrid feature vectors.

Table 7: Classifier Performance Comparison

Model	Accuracy	Weighted F1 Score	Training Time
SVM	**50.14%**	**50.13%**	37.01 sec
KNN	38.71%	37.02%	0.01 sec

Model	Accuracy	Weighted F1 Score	Training Time
Random Forest	39.14%	38.50%	25.70 sec

Analysis

From Table 7:

Support Vector Machine

Advantages:

- highest classification accuracy,
- best weighted F1 score,
- good class boundary separation,
- effective in high-dimensional feature space.

Limitation:

- higher training time.

K-Nearest Neighbors

Advantages:

- extremely fast training,
- simple implementation.

Limitation:

- lower accuracy,
- computationally expensive inference,
- affected by high-dimensional feature vectors.

Random Forest

Advantages:

- moderate performance,
- robust ensemble method,
- handles nonlinear boundaries.

Limitation:

- lower accuracy than SVM,
- relatively large training overhead.

Observation

The hybrid HOG–LBP feature vector is high-dimensional (4036 features), and **SVM is particularly well suited for such feature spaces**, resulting in the best performance.

6.4 Best Model Selection

Based on quantitative evaluation:

🏆 **Best Model = Support Vector Machine (SVM)**

Reason for selection:

- highest accuracy,
- highest F1 score,
- balanced classification performance,
- strong generalization capability.

6.5 Confusion Matrix Analysis

A confusion matrix was generated for the best-performing SVM classifier to understand class-wise prediction behavior. Your notebook output includes this visualization.

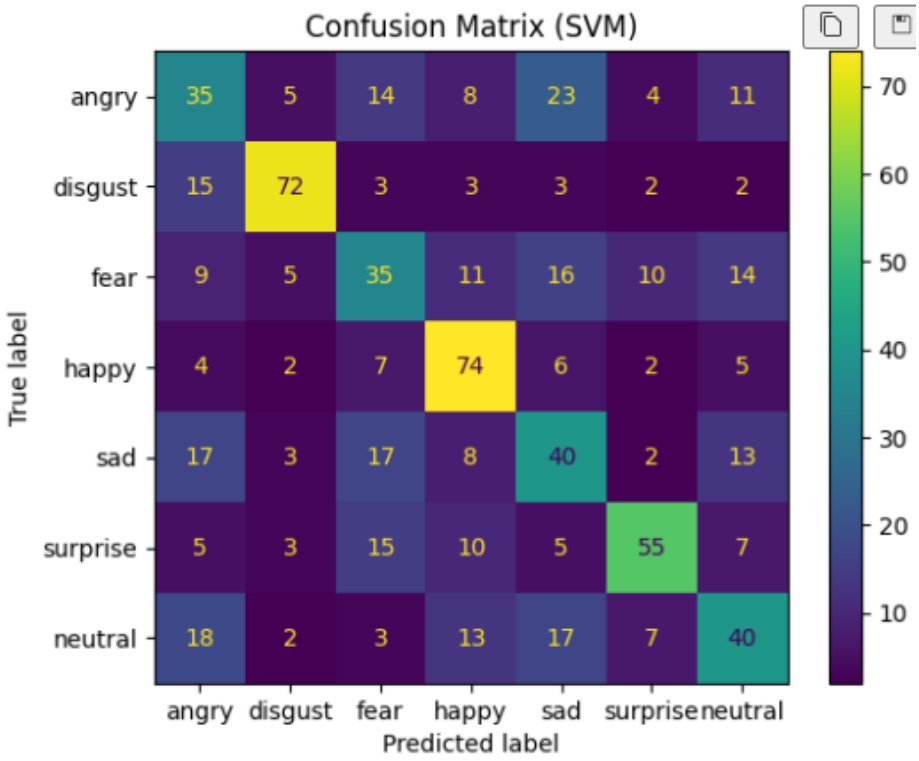


Figure 6.1: Confusion Matrix

Interpretation

The confusion matrix indicates:

- certain emotions such as **Happy** and **Surprise** are comparatively easier to classify,
- classes like **Fear**, **Sad**, and **Neutral** show overlap,
- **Disgust**, being underrepresented in the dataset, may exhibit inconsistent prediction performance.

This is expected due to class imbalance and facial similarity among emotions.

6.6 Precision–Recall Analysis

Precision–Recall curves were plotted for all seven emotion classes.

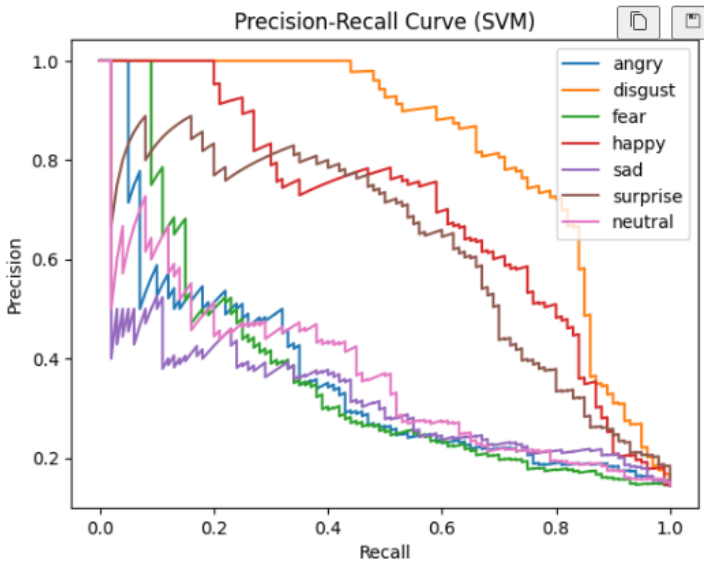


Figure 6.2: Precision–Recall Curve

Interpretation

Precision–Recall analysis provides:

- insight into class-wise discrimination,
- confidence distribution,
- classifier reliability across thresholds.

Higher PR area indicates better separability of emotional classes.

Classes with distinct facial muscle patterns generally produce better PR curves.

6.7 ROC Curve Analysis

Receiver Operating Characteristic (ROC) curves were generated for multi-class classification using

one-vs-rest evaluation.

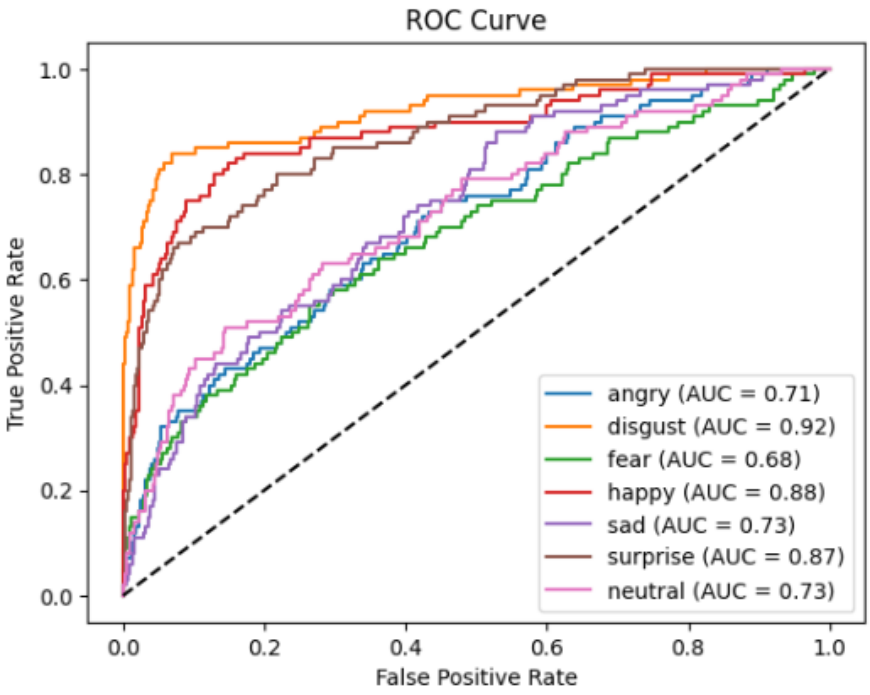


Figure 6.3: ROC Curve

Interpretation

ROC analysis indicates:

- classifier discrimination capability,
- true positive vs false positive trade-off,
- separability of individual emotion classes.

Curves closer to the top-left corner indicate stronger classification performance.

6.8 Random Prediction Testing

The trained SVM classifier was tested on unseen random images from the FER2013 test set.

Prediction outputs included:

- actual class,
- predicted class,
- correctness indication.

Examples observed in your notebook:

Actual	Predicted	Result
Disgust	Disgust	Correct

Actual	Predicted	Result
Neutral	Sad	Wrong
Neutral	Neutral	Correct

This demonstrates realistic model behavior, including both correct classifications and occasional misclassifications.

Figure 6.4: Sample Prediction Output

6.9 Key Findings

Major findings of the experimental study:

1. Preprocessing significantly improved feature quality.
2. HOG effectively captured facial structure.
3. LBP effectively captured local texture patterns.
4. Hybrid feature fusion improved representation.
5. Feature scaling improved classifier convergence.
6. SVM delivered the best overall performance.
7. FER remains challenging due to overlapping expressions.

---

7 – DISCUSSION

7.1 Performance Discussion

The proposed hybrid feature engineering approach demonstrated moderate but meaningful classification performance for seven-class facial emotion recognition.

The achieved accuracy of **50.14%** significantly exceeds the baseline (**14.28%**), confirming the effectiveness of:

- preprocessing,
- handcrafted features,

- classifier selection.

7.2 Why HOG + LBP Worked Well

HOG contribution:

Captured:

- eyebrow shape,
- mouth curvature,
- cheek contours,
- forehead wrinkles.

LBP contribution:

Captured:

- micro texture changes,
- local facial muscle activation,
- skin texture transitions,
- subtle emotional patterns.

Combined representation provided complementary information.

7.3 Limitations

The project has several limitations:

1) Dataset imbalance

Disgust class has very few samples.

2) Low-resolution images

FER2013 images are noisy.

3) Facial overlap

Sad / Neutral / Fear have similar patterns.

4) No deep semantic learning

Handcrafted features have representational limitations.

5) Static image only

Temporal emotion evolution is ignored.

7.4 Practical Applications

This work can be applied in:

- online learning engagement monitoring,
- mental health support systems,
- smart surveillance,
- customer sentiment analysis,
- driver fatigue monitoring,
- interactive AI assistants.

---

8 – CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project presented a complete **Facial Emotion Recognition system using Hybrid HOG–LBP feature extraction and Machine Learning classification**.

Major stages included:

- dataset preparation,
- preprocessing,
- handcrafted feature extraction,
- feature fusion,
- scaling,
- model training,
- evaluation,
- prediction.

Experimental results showed:

- handcrafted hybrid descriptors are effective,
- preprocessing substantially improves performance,
- SVM is best among tested classifiers,
- emotion recognition remains challenging in unconstrained datasets.

Final best performance:

Accuracy = 50.14%
Weighted F1 = 50.13%

The project successfully demonstrates a practical FER pipeline using computer vision and machine learning.

8.2 Future Work

Future enhancements may include:

1. Deep CNN-based feature extraction.
2. Transfer learning with pretrained networks.
3. Facial landmark detection.
4. Attention-based feature learning.
5. Video emotion recognition.
6. Real-time webcam deployment.
7. Multimodal sentiment fusion (speech + image).
8. Dataset balancing through augmentation.
9. Edge deployment optimization.
10. Explainable AI visualization.

REFERENCES *(IEEE Style)*

[1] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," *Proc. IEEE CVPR*, 2005.

[2] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns," *IEEE TPAMI*, 2002.

[3] I. Goodfellow et al., "Challenges in Representation Learning: A Report on FER2013," *ICML Workshop*, 2013.

[4] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.

[6] C. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[7] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2011.

[8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson, 2021.

APPENDIX A – SOURCE CODE

```
# ============================================================
# STEP 0: IMPORTS
# ============================================================
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix,
ConfusionMatrixDisplay
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.utils import shuffle
import pickle
# ============================================================
# PARAMETERS
# ============================================================
IMG_SIZE = 128
TRAIN_DIR = "/kaggle/input/datasets/msambare/fer2013/train"
TEST_DIR = "/kaggle/input/datasets/msambare/fer2013/test"

CATEGORIES = ["angry", "disgust", "fear", "happy", "sad", "surprise",
"neutral"]
# ============================================================
# STEP 1: DATA COLLECTION
# ============================================================

def load_limited_data(folder, limit):
    data, labels = [], []
    count_dict = {}
    print(f"\n🗁 Loading from: {folder}")


    for label, category in enumerate(CATEGORIES):
        path = os.path.join(folder, category)
        print(f"→ Processing class: {category}")

        images = os.listdir(path)[:limit]
        count_dict[category] = len(images)
```

```
        for img_name in images:
            img_path = os.path.join(path, img_name)
            img = cv2.imread(img_path)
            if img is None: continue

            data.append(img)
            labels.append(label)

    return np.array(data), np.array(labels), count_dict


print("\n📊 Loading dataset...")
X_train_raw, y_train, train_count = load_limited_data(TRAIN_DIR, 500)
X_test_raw, y_test, test_count = load_limited_data(TEST_DIR, 100)

# Plot class distribution
def count_full_dataset(folder):
    count_dict = {}

    for category in CATEGORIES:
        path = os.path.join(folder, category)

        # count all images
        count = len(os.listdir(path))
        count_dict[category] = count

    return count_dict

# Count FULL dataset (not limited)
train_full_count = count_full_dataset(TRAIN_DIR)
test_full_count = count_full_dataset(TEST_DIR)

print("Full Train Count:", train_full_count)
print("Full Test Count:", test_full_count)

def plot_distribution(count_dict, title):
    plt.bar(count_dict.keys(), count_dict.values())
    plt.title(title)
    plt.xticks(rotation=45)
    plt.show()

plot_distribution(train_full_count, "Train Data Distribution")
plot_distribution(test_full_count, "Test Data Distribution")

# Show sample images
def show_samples(X, y):
    plt.figure(figsize=(10,5))
    for i in range(7):
        idx = np.where(y == i)[0][0]
```

```
        plt.subplot(2,4,i+1)
        plt.imshow(cv2.cvtColor(X[idx], cv2.COLOR_BGR2RGB))
        plt.title(CATEGORIES[i])
        plt.axis('off')
    plt.show()

show_samples(X_train_raw, y_train)


# ============================================================
# STEP 2: PREPROCESSING
# ============================================================

def preprocess_image(img, visualize=False):
    original = img.copy()

    # Resize
    resized = cv2.resize(img, (IMG_SIZE, IMG_SIZE))

    # Denoise
    denoised = cv2.GaussianBlur(resized, (5,5), 0)

    # Enhance
    # Enhance (FIXED)
    if len(denoised.shape) == 3:
        gray = cv2.cvtColor(denoised, cv2.COLOR_BGR2GRAY)
    else:
        gray = denoised
    clahe = cv2.createCLAHE(clipLimit=2.0)
    enhanced = clahe.apply(gray)

    if visualize:
        titles = ["Original", "Resized", "Denoised", "Enhanced"]
        images = [original, resized, denoised, enhanced]

        plt.figure(figsize=(10,4))
        for i in range(4):
            plt.subplot(1,4,i+1)
            if i<3:
                plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
            else:
                plt.imshow(images[i], cmap='gray')
            plt.title(titles[i])
            plt.axis('off')
        plt.show()

    return enhanced
```

```
# Apply preprocessing
print("\n⚙ Preprocessing...")
X_train = np.array([preprocess_image(img) for img in X_train_raw])
X_test = np.array([preprocess_image(img) for img in X_test_raw])

# Visualize one sample
preprocess_image(X_train_raw[0], visualize=True)

# ============================================================
# STEP 3: FEATURE EXTRACTION (FINAL CLEAN VERSION)
# ============================================================

hog = cv2.HOGDescriptor()

# ------------------------------
# DISPLAY: SIDE-BY-SIDE
# ------------------------------
def show_comparison(title1, img1, title2, img2, cmap='gray'):
    plt.figure(figsize=(8,3))

    plt.subplot(1,2,1)
    plt.imshow(img1, cmap=cmap)
    plt.title(title1)
    plt.axis('off')

    plt.subplot(1,2,2)
    plt.imshow(img2, cmap=cmap)
    plt.title(title2)
    plt.axis('off')

    plt.suptitle("Before vs After")
    plt.show()


# ------------------------------
# SAMPLE IMAGE
# ------------------------------
sample_img = X_train[0]

print("\n🔍 FEATURE EXTRACTION VISUALIZATION")

# ============================================================
# 1. HOG FEATURE
# ============================================================

print("\n HOG Feature Extraction")

hog_input = cv2.resize(sample_img, (64, 128))
```

```
show_comparison(
    "Original Image",
    sample_img,
    "Resized for HOG",
    hog_input
)

hog_feat_sample = hog.compute(hog_input)
print("HOG Feature Size:", hog_feat_sample.shape)


# ============================================================
# 2. LBP FEATURE
# ============================================================

print("\n LBP Feature Extraction")

def compute_lbp_fast(img):
    center = img[1:-1,1:-1]

    lbp = (
        (img[:-2,:-2] > center) << 7 |
        (img[:-2,1:-1] > center) << 6 |
        (img[:-2,2:] > center) << 5 |
        (img[1:-1,2:] > center) << 4 |
        (img[2:,2:] > center) << 3 |
        (img[2:,1:-1] > center) << 2 |
        (img[2:,:-2] > center) << 1 |
        (img[1:-1,:-2] > center)
    )

    return lbp

lbp_img = compute_lbp_fast(sample_img)

show_comparison(
    "Original Image",
    sample_img,
    "LBP Image",
    lbp_img
)

lbp_hist_sample, _ = np.histogram(lbp_img.ravel(), bins=256, range=(0,256))
lbp_hist_sample = lbp_hist_sample / (lbp_hist_sample.sum() + 1e-6)

print("LBP Feature Size:", lbp_hist_sample.shape)


# ============================================================
```

```
# FINAL FEATURE EXTRACTION FUNCTION (ONLY HOG + LBP)
# ============================================================

def extract_features(img):

    # HOG
    hog_feat = hog.compute(cv2.resize(img, (64,128))).flatten()

    # LBP
    lbp_img = compute_lbp_fast(img)
    lbp_hist, _= np.histogram(lbp_img.ravel(), bins=256, range=(0,256))
    lbp_hist = lbp_hist / (lbp_hist.sum() + 1e-6)

    # Combine
    return np.hstack([hog_feat, lbp_hist])


# ============================================================
# APPLY TO DATASET
# ============================================================

print("\n Extracting features for dataset...")

X_train_feat = []
for i, img in enumerate(X_train):
    if i % 200 == 0:
        print(f"Processing {i}/{len(X_train)}")

    X_train_feat.append(extract_features(img))

X_train_feat = np.array(X_train_feat)

X_test_feat = np.array([extract_features(img) for img in X_test])


# ============================================================
# FEATURE SIZE
# ============================================================

print("\n Final Feature Vector Size:", X_train_feat.shape[1])


# ============================================================
# FEATURE COMPARISON TABLE (UPDATED)
# ============================================================

feature_sizes = {
    "HOG": hog.compute(np.zeros((128,64), dtype=np.uint8)).shape[0],
    "LBP": 256
```

```
}

print("\n📊 Feature Comparison Table:")
print(pd.DataFrame(list(feature_sizes.items()),
columns=["Feature","Dimension"]))

# ============================================================
# STEP 4: ALGORITHM IMPLEMENTATION (UPDATED)
# ============================================================

import time
from sklearn.dummy import DummyClassifier

# ------------------------------
# SCALING
# ------------------------------
scaler = StandardScaler()
X_train_feat = scaler.fit_transform(X_train_feat)
X_test_feat = scaler.transform(X_test_feat)
# 🔥 SAVE SCALER (ADD THIS)
import pickle

with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

print("✅ Scaler saved successfully!")

# ------------------------------
# BASELINE (BEFORE TRAINING)
# ------------------------------
print("\n📊 Baseline Model (Before Training)")

dummy = DummyClassifier(strategy="most_frequent")
dummy.fit(X_train_feat, y_train)

y_dummy = dummy.predict(X_test_feat)

baseline_acc = accuracy_score(y_test, y_dummy)
baseline_f1 = f1_score(y_test, y_dummy, average='weighted')

print("Baseline Accuracy:", baseline_acc)
print("Baseline F1 Score:", baseline_f1)


# ------------------------------
# MODELS
# ------------------------------
models = {
    "SVM": SVC(
```

```
        kernel='rbf',
        C=10,
        gamma='scale',
        class_weight='balanced'),
    "KNN": KNeighborsClassifier(),
    "RandomForest": RandomForestClassifier()
}

results = {}

# ------------------------------
# TRAINING LOOP
# ------------------------------
for name, model in models.items():
    print(f"\n🚀 Training {name}...")

    start_time = time.time()

    model.fit(X_train_feat, y_train)

    end_time = time.time()
    training_time = end_time - start_time

    y_pred = model.predict(X_test_feat)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')

    results[name] = [acc, f1, training_time]

    # Save model
    with open(f"{name}_model.pkl", "wb") as f:
        pickle.dump(model, f)

    print(f"{name} Accuracy: {acc:.4f}")
    print(f"{name} F1 Score: {f1:.4f}")
    print(f"{name} Training Time: {training_time:.2f} seconds")


# ------------------------------
# RESULTS TABLE
# ------------------------------
import pandas as pd

df = pd.DataFrame(results, index=["Accuracy", "F1 Score", "Training Time"]).T

print("\n📊 Final Comparison Table:")
print(df)
```

```
# ============================================================
# STEP 5: EVALUATION (FIXED)
# ============================================================

import pandas as pd
from sklearn.metrics import precision_recall_curve
from sklearn.preprocessing import label_binarize

# ------------------------------
# SELECT BEST MODEL (BASED ON ACCURACY)
# ------------------------------
best_model_name = max(results, key=lambda x: results[x][0])
print("\n🏆 Best Model:", best_model_name)

best_model = models[best_model_name]


# ------------------------------
# CONFUSION MATRIX
# ------------------------------
y_pred = best_model.predict(X_test_feat)

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=CATEGORIES)

disp.plot()
plt.title(f"Confusion Matrix ({best_model_name})")
plt.show()


# ------------------------------
# PRECISION-RECALL CURVE
# ------------------------------
print("\n📈 Plotting Precision-Recall Curve...")

# Convert labels to binary
y_test_bin = label_binarize(y_test, classes=range(len(CATEGORIES)))

# Check if model supports decision_function or predict_proba
if hasattr(best_model, "decision_function"):
    scores = best_model.decision_function(X_test_feat)
elif hasattr(best_model, "predict_proba"):
    scores = best_model.predict_proba(X_test_feat)
else:
    print("❌ PR curve not supported for this model")
    scores = None

# Plot PR curve
if scores is not None:
```

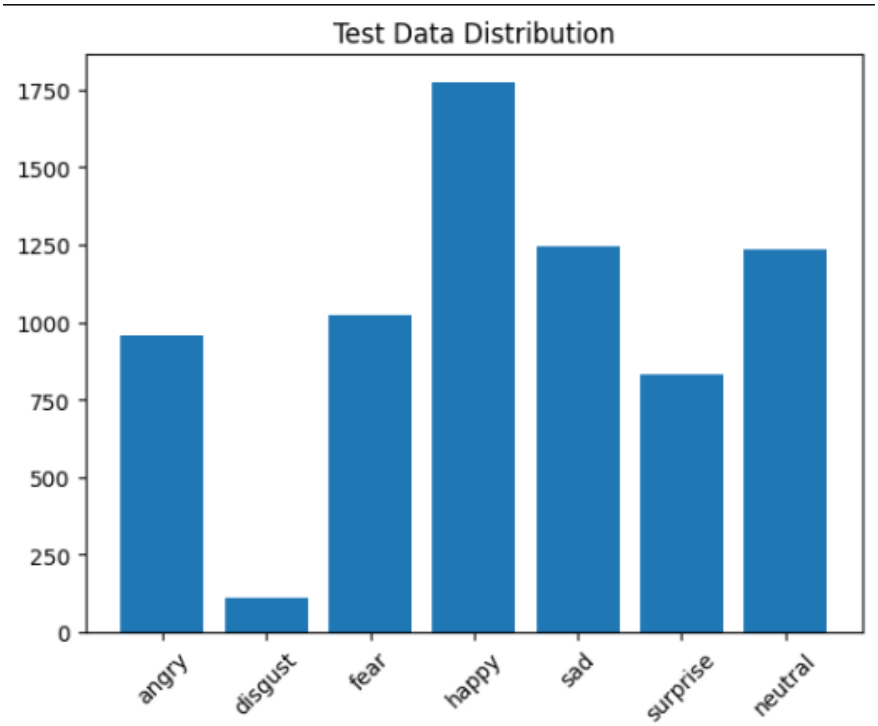
```
    for i in range(len(CATEGORIES)):
        precision, recall, _ = precision_recall_curve(y_test_bin[:, i],
scores[:, i])
        plt.plot(recall, precision, label=CATEGORIES[i])

    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title(f"Precision-Recall Curve ({best_model_name})")
    plt.legend()
    plt.show()

# ------------------------------
# ROC CURVE
# ------------------------------
from sklearn.metrics import roc_curve, auc

# Binarize labels
y_test_bin = label_binarize(y_test, classes=range(len(CATEGORIES)))

# Get scores
if hasattr(best_model, "predict_proba"):
    scores = best_model.predict_proba(X_test_feat)
else:
    scores = best_model.decision_function(X_test_feat)

# Plot ROC
for i in range(len(CATEGORIES)):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], scores[:, i])
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr, label=f"{CATEGORIES[i]} (AUC = {roc_auc:.2f})")

plt.plot([0,1], [0,1], 'k--')  # random line
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

# ============================================================
# TEST MODEL ON RANDOM NEW IMAGE
# ============================================================

import random
import os
import cv2
import matplotlib.pyplot as plt
```

```
def predict_random_image():


    # Pick random class
    random_class = random.choice(CATEGORIES)

    class_path = os.path.join(TEST_DIR, random_class)

    # Pick random image
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)

    print(f"\n🔍 Testing Image: {img_path}")
    print(f"Actual Label: {random_class}")

    # ------------------------------
    # LOAD IMAGE
    # ------------------------------
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("❌ Error loading image")
        return

    # PREPROCESS (VERY IMPORTANT)
    img_processed = preprocess_image(img)

    # FEATURE EXTRACTION (HOG + LBP)
    features = extract_features(img_processed)
    features = features.reshape(1, -1)

    # SCALE
    features = scaler.transform(features)

    # PREDICT
    pred = best_model.predict(features)[0]

    predicted_label = CATEGORIES[pred]

    # DISPLAY RESULT
    plt.imshow(img_processed, cmap='gray')
    plt.title(f"Predicted: {predicted_label} | Actual: {random_class}")
    plt.axis('off')
    plt.show()
    print("🎯 Predicted:",predicted_label)
    print("📌 Actual:", random_class)
```

```
    # CHECK CORRECT / WRONG
    if predicted_label == random_class:
        print("✅ Correct Prediction")
    else:
        print("❌ Wrong Prediction")

for _ in range(5):
    predict_random_image()
```

APPENDIX B – OUTPUT SCREENSHOTS

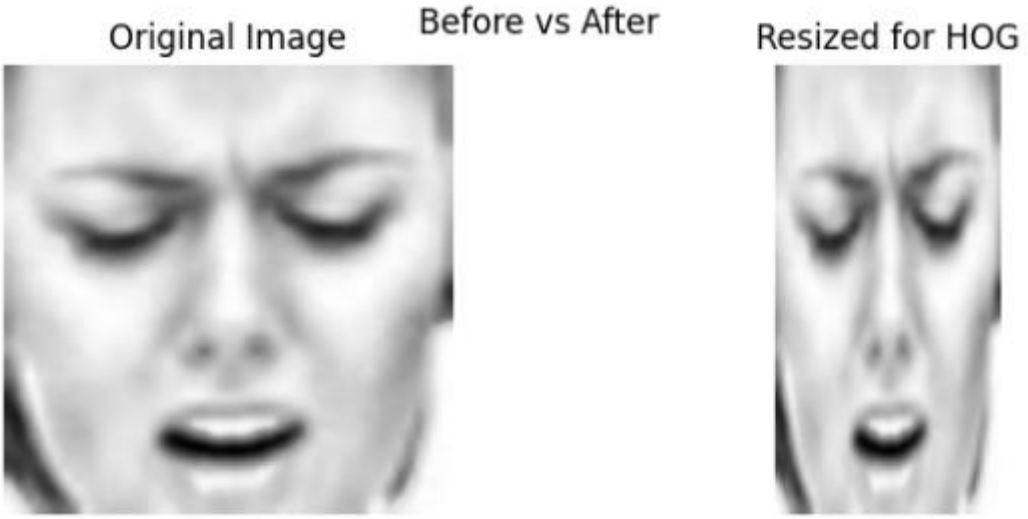
1.Dataset Distribution Chart



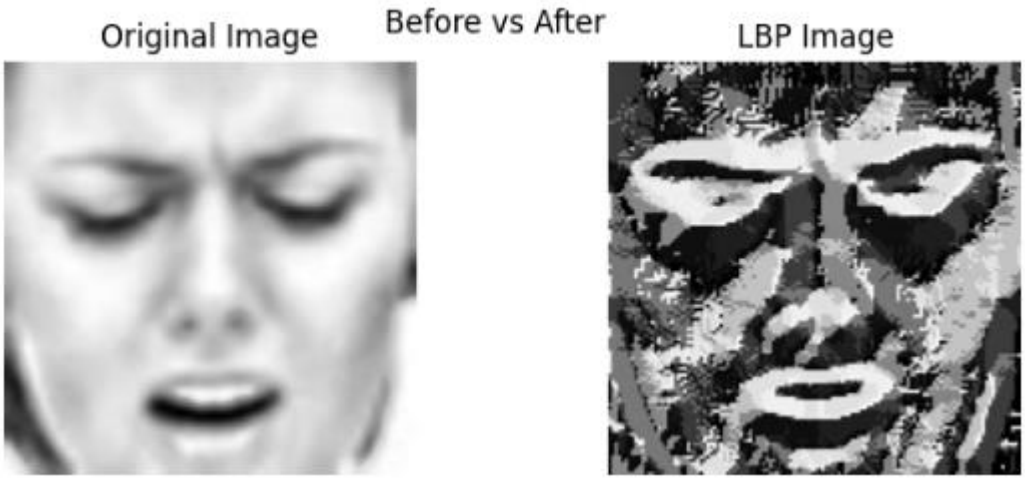
2.Sample Dataset Images

3.Preprocessing Pipeline

4.HOG Visualization



5.LBP Visualization

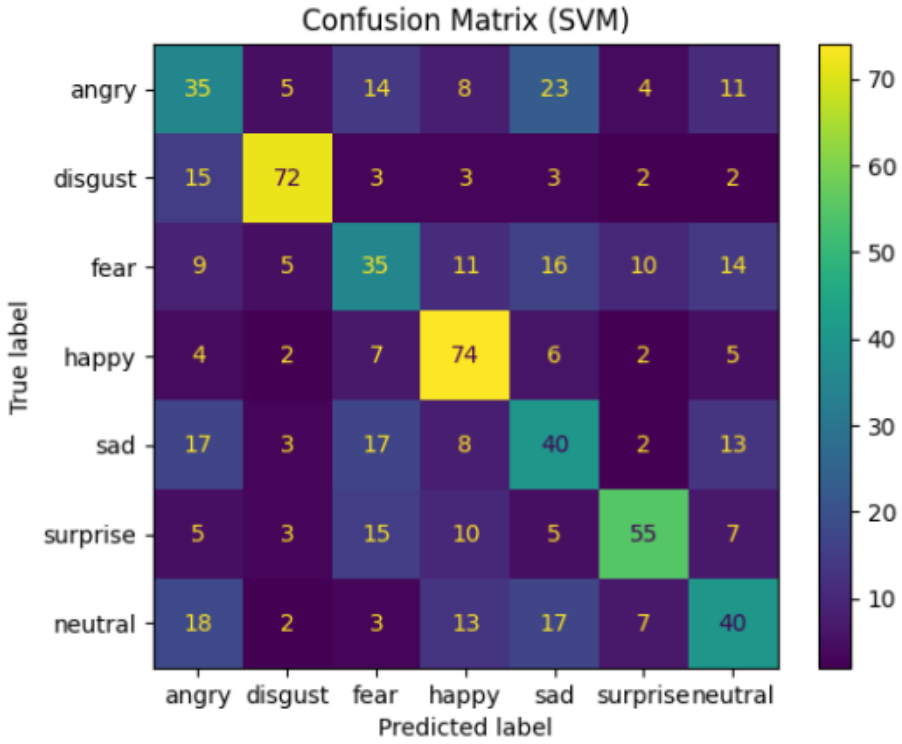


6.Model Comparison Table

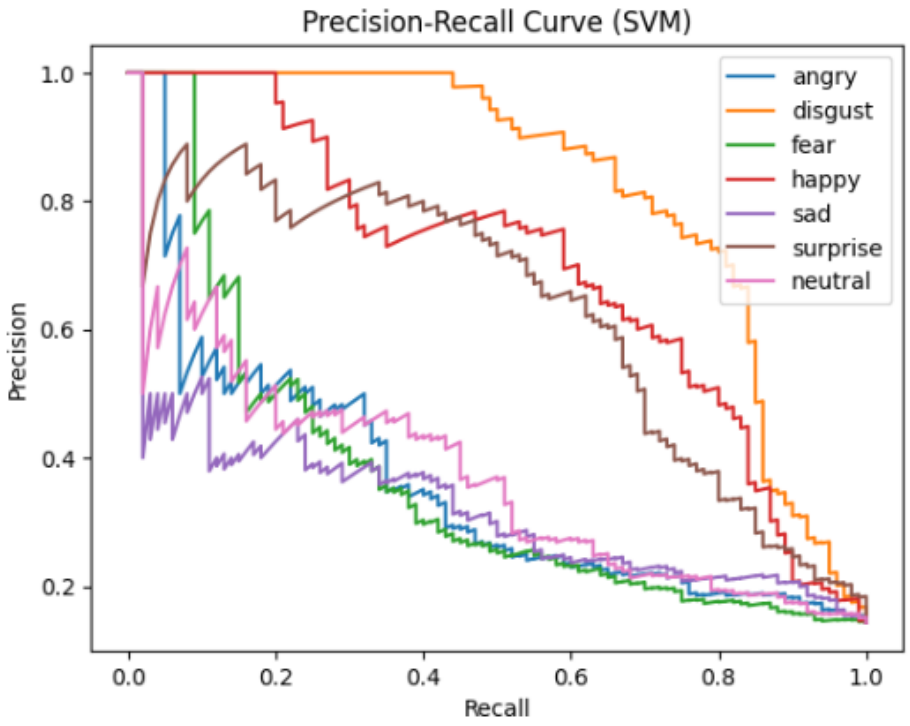
📊 Final Comparison Table:

	Accuracy	F1 Score	Training Time
SVM	0.501429	0.501378	37.013706
KNN	0.387143	0.370231	0.010188
RandomForest	0.391429	0.385049	25.699755

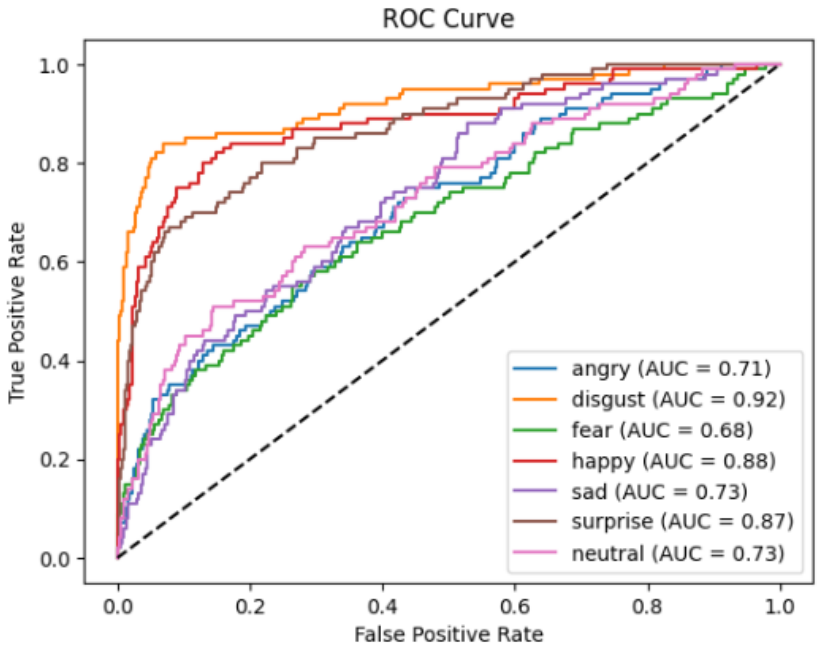
7.Confusion Matrix



8.Precision–Recall Curve



9.ROC Curve



10.Final Prediction Output